

2.1 Types

[This section corresponds to K&R Sec. 2.2]

There are only a few basic data types in C. The first ones we'll be encountering and using are:

- `char` a character
- `int` an integer, in the range -32,767 to 32,767
- `long int` a larger integer (up to +-2,147,483,647)
- `float` a floating-point number
- `double` a floating-point number, with more precision and perhaps greater range than `float`

If you can look at this list of basic types and say to yourself, "Oh, how simple, there are only a few types, I won't have to worry much about choosing among them," you'll have an easy time with declarations. (Some masochists wish that the type system were more complicated so that they could specify more things about each variable, but those of us who would rather not have to specify these extra things each time are glad that we don't have to.)

The ranges listed above for types `int` and `long int` are the guaranteed minimum ranges. On some systems, either of these types (or, indeed, any C type) may be able to hold larger values, but a program that depends on extended ranges will not be as portable. Some programmers become obsessed with knowing exactly what the sizes of data objects will be in various situations, and go on to write programs which depend on these exact sizes. Determining or controlling the size of an object is occasionally important, but most of the time we can sidestep size issues and let the compiler do most of the worrying.

(From the ranges listed above, we can determine that type `int` must be at least 16 bits, and that type `long int` must be at least 32 bits. But neither of these sizes is exact; many systems have 32-bit `ints`, and some systems have 64-bit `long ints`.)

You might wonder how the computer stores characters. The answer involves a *character set*, which is simply a mapping between some set of characters and some set of small numeric codes. Most machines today use the ASCII character set, in which the letter A is represented by the code 65, the ampersand & is represented by the code 38, the digit 1 is represented by the code 49, the space character is represented by the code 32, etc. (Most of the time, of course, you have *no* need to know or even worry about these particular code values; they're automatically translated into the right shapes on the screen or printer when characters are printed out, and they're automatically generated when you type characters on the keyboard. Eventually, though, we'll appreciate, and even take some control over, exactly when these translations--from characters to their numeric codes--are performed.) Character codes are usually small--the largest code value in ASCII is 126, which is the ~ (tilde or circumflex) character. Characters usually fit in a byte, which is usually 8 bits. In C, type `char` is defined as occupying one byte, so it is usually 8 bits.

Most of the simple variables in most programs are of types `int`, `long int`, or `double`. Typically, we'll use `int` and `double` for most purposes, and `long int` any time we need to hold integer values greater than 32,767. As we'll see, even when we're manipulating individual characters, we'll usually use an `int` variable, for reasons to be discussed later. Therefore, we'll rarely use individual variables of type `char`; although we'll use plenty of arrays of `char`.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)